Runge-Kutta-Solver and Step Size Control

Elizabeth Klioner M.Sc., Niklas Meyners M.Sc. | Chair of Space Technology | Institute of Aeronautics and Astronautics (ILR)

Introduction

- Typically, numerically solving the Initial Value Problem for a given initial state with the Equation of Motion is used for high-precision orbit propagation
- For this a sufficiently accurate numerical method is necessary
- One kind of methods are single-step methods, to which Runge-Kutta methods belong
- Computational effort vs. accuracy of a method is essential
- Implementing Runge-Kutta methods efficiently is a challenge of itself

Runge-Kutta Methods Recap (1)

$$\boldsymbol{x}_{i+1}^{h} = \boldsymbol{x}_{i}^{h} + h\sum_{j=1}^{m}\beta_j \boldsymbol{k}_j, \quad \boldsymbol{k}_j = \boldsymbol{f}\left(t_i + \varrho_j h, x_i^h + h\sum_{l=1}^{m}\gamma_{jl}\boldsymbol{k}_l\right), j = 1, \dots, m$$

General form of a m stage Runge-Kutta Method

- The term updating $\boldsymbol{x}_i^h$ contains the increment function $\boldsymbol{\Phi}$:

$$\boldsymbol{x}_{i+1}^{h} = \boldsymbol{x}_{i}^{h} + h\boldsymbol{\Phi}(t_i, \boldsymbol{x}_i, h)$$

- The superscript h denotes the dependence on the step size and is omitted in the following
- β, ϱ, γ are often written as c, a, b respectively (check naming when you look up coefficients)
- How can the accuracy of a Runge-Kutta method be controlled?

Runge-Kutta Methods Recap (2)

- Coefficients ϱ, γ, β are often expressed in Butcher tableaus in compact form

$$\begin{array}{c|ccc} \varrho_1 & \gamma_{11} & \cdots & \gamma_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ \varrho_m & \gamma_{m1} & \cdots & \gamma_{mm} \\ \hline & \beta_1 & \cdots & \beta_m \end{array} \qquad \begin{array}{c|c} \boldsymbol{\varrho} & \boldsymbol{\Gamma} \\ \hline & \boldsymbol{\beta}^T \end{array}$$

- Coefficients for most methods are listed in Butcher tableaus in literature
- We will only focus on explicit methods, i.e., $\varrho_1 = 0$ and $\gamma_{ij} = 0$ for $\mathrm{i} \leq j$
  - Implicit methods involve extra steps to solve the linear equations for unknown coefficients
  - Explicit methods are fitting for application in orbital mechanics

Runge-Kutta Methods Recap (3)

- Comparing RK4 and RKF45
- What do you notice?

0	0	0	0	0
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0
$\frac{1}{2}$	0	$\frac{1}{2}$	0	0
1	0	0	1	0
	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$

0	0	0	0	0	0	0	
$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$	0	0	0	0	zusätzliche Spalte bei RKF5
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$	0	0	0	
1	$\frac{439}{216}$	-8	$\frac{3680}{513}$	$-\frac{845}{4104}$	0	0	
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	0	zusätzliche Zeile bei RKF5
	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$		β_i für RKF4
	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$	β_i für RKF5

[1]

Runge-Kutta Methods Recap (4)

- With the given coefficients the equations are simple to write down

0	0	0	0	0	ϱ_1	$\gamma_{1,1}$	$\gamma_{1,2}$	$\gamma_{1,3}$	$\gamma_{1,4}$
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0	ϱ_2	$\gamma_{2,1}$	$\gamma_{2,2}$	$\gamma_{2,3}$	$\gamma_{2,4}$
$\frac{1}{2}$	0	$\frac{1}{2}$	0	0	ϱ_3	$\gamma_{3,1}$	$\gamma_{3,2}$	$\gamma_{3,3}$	$\gamma_{3,4}$
1	0	0	1	0	ϱ_4	$\gamma_{4,1}$	$\gamma_{4,2}$	$\gamma_{4,3}$	$\gamma_{4,4}$
	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$		β_1	β_2	β_3	β_4

$$\boldsymbol{k}_1 = \boldsymbol{f}\left(t_i + \varrho_1 h, \boldsymbol{x}_i + \gamma_{1,1} h\boldsymbol{k}_1 + \gamma_{1,2} h\boldsymbol{k}_2 + \gamma_{1,3} h\boldsymbol{k}_3 + \gamma_{1,4} h\boldsymbol{k}_4\right) = \boldsymbol{f}(t_i, \boldsymbol{x}_i),$$

$$\boldsymbol{k}_2 = \boldsymbol{f}\left(t_i + \varrho_2 h, \boldsymbol{x}_i + \gamma_{2,1} h\boldsymbol{k}_1 + \gamma_{2,2} h\boldsymbol{k}_2 + \gamma_{2,3} h\boldsymbol{k}_3 + \gamma_{2,4} h\boldsymbol{k}_4\right) = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_1\right)$$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(t_i + \varrho_3 h, \boldsymbol{x}_i + \gamma_{3,1} h\boldsymbol{k}_1 + \gamma_{3,2} h\boldsymbol{k}_2 + \gamma_{3,3} h\boldsymbol{k}_3 + \gamma_{3,4} h\boldsymbol{k}_4\right) = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_2\right)$$

Runge-Kutta Methods Recap (5)

- With the given coefficients the equations are simple to write down

0	0	0	0	0	ϱ_1	$\gamma_{1,1}$	$\gamma_{1,2}$	$\gamma_{1,3}$	$\gamma_{1,4}$
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0	ϱ_2	$\gamma_{2,1}$	$\gamma_{2,2}$	$\gamma_{2,3}$	$\gamma_{2,4}$
$\frac{1}{2}$	0	$\frac{1}{2}$	0	0	ϱ_3	$\gamma_{3,1}$	$\gamma_{3,2}$	$\gamma_{3,3}$	$\gamma_{3,4}$
1	0	0	1	0	ϱ_4	$\gamma_{4,1}$	$\gamma_{4,2}$	$\gamma_{4,3}$	$\gamma_{4,4}$
	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$		β_1	β_2	β_3	β_4

$$\boldsymbol{k}_4 = \boldsymbol{f}\left(t_i + \varrho_4 h, \boldsymbol{x}_i + \gamma_{4,1} h\boldsymbol{k}_1 + \gamma_{4,2} h\boldsymbol{k}_2 + \gamma_{4,3} h\boldsymbol{k}_3 + \gamma_{4,4} h\boldsymbol{k}_4\right) = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_2\right)$$

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + h\boldsymbol{\Phi}(t_i, \boldsymbol{x}_i, h) = \boldsymbol{x}_i + h(\beta_1\boldsymbol{k}_1 + \beta_2\boldsymbol{k}_2 + \beta_3\boldsymbol{k}_3 + \beta_4\boldsymbol{k}_4) = \boldsymbol{x}_i + \frac{1}{6}h(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4)$$

Runge-Kutta Methods Recap (6)

- For more complex coefficient sets like RKF45 it works exactly the same
- Other than RK4, RKF45 uses all slopes $\boldsymbol{k}_{i-1}$ for the calculation of $\boldsymbol{k}_i$
- The number of stages (function evaluations) is visible in the dimensions of the matrix $\boldsymbol{\Gamma}$ with $\boldsymbol{\Gamma} \in \mathbb{R}^{6\times6}$ for the 5th order method and $\boldsymbol{\Gamma} \in \mathbb{R}^{5\times5}$ for the 4th order method

0	0	0	0	0	0	0	
$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$	0	0	0	0	zusätzliche Spalte bei RKF5
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$	0	0	0	
1	$\frac{439}{216}$	-8	$\frac{3680}{513}$	$-\frac{845}{4104}$	0	0	
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	0	zusätzliche Zeile bei RKF5
	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$		β_i für RKF4
	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$	β_i für RKF5

[1]

Runge-Kutta Methods Recap (7)

- Order p and number of evaluations per step m differ greatly the higher p is

p	1	2	3	4	5	6	7	8	9	10	12	14
m_p	1	2	3	4	6	7	9	11	15	17	25	35

[1]

- For the embedded RKF45 method(s), the 4th order method needs 5 evaluations but shares all these with the 5th order method → very good for step size control
- If no step size control is used (which should be used, especially for orbital mechanics) only four evaluations are necessary for a 4th order method (see RK4)
- What is the computationally expensive part of a Runge-Kutta Method?

Exercise 1:

- Think about how to implement an explicit Runge-Kutta stepper that is
  1. Not hardcoded for a specific method
  2. Flexible w.r.t. the order of the method
  3. Could potentially save intermediate steps (e.g., for interpolation)
- Which steps are necessary for an explicit RK stepper?
- Which information (about the order and about the solution) is needed, which is nice to have?
- Hint: for-loops might not be avoidable, keep in mind what the computationally expensive part is (function evaluation of the provided ODE)

Exercise 1: Solution

- Using different coefficient sets implies that the number of stages differs
  - For-loop for the computation of all stages in a time step is not avoidable
- For the computation of the variables that are needed for the function evaluation, a for-loop is avoidable
  - By storing $\boldsymbol{k}_i$ as column vectors in a matrix, matrix multiplication with the correct submatrix of $\boldsymbol{\Gamma}$ will automatically yield the correct variable

Exercise 1: Solution

- As example, consider the third stage of RK4

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(t_i + \varrho_3 h, \boldsymbol{x}_i + \gamma_{3,1} h\boldsymbol{k}_1 + \gamma_{3,2} h\boldsymbol{k}_2 + \gamma_{3,3} h\boldsymbol{k}_3 + \gamma_{3,4} h\boldsymbol{k}_4\right) = \boldsymbol{f}\left(t_i + \frac{1}{2}h, \boldsymbol{x}_i + \frac{1}{2}h\,\boldsymbol{k}_2\right)$$

- To avoid unnecessary operations and using the fact that RK4 is an explicit method, the terms containing $\boldsymbol{k}_3$ and $\boldsymbol{k}_4$ can be ignored through clever array slices of the matrices $\boldsymbol{K}$ (storing the slopes as column vectors) and $\boldsymbol{\Gamma}$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(t_i + \varrho_3 h, \boldsymbol{x}_i + \gamma_{3,1} h\boldsymbol{k}_1 + \gamma_{3,2} h\boldsymbol{k}_2\right)$$

Exercise 1: Solution

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(t_i + \varrho_3 h, \boldsymbol{x}_i + \gamma_{3,1} h \boldsymbol{k}_1 + \gamma_{3,2} h \boldsymbol{k}_2\right)$$

- Since this is the third stage, the submatrix of $\boldsymbol{\Gamma}$ is easily visible as $[\gamma_{3,1} \quad \gamma_{3,2}]$ and the computation of the second variable simplifies to

$$\boldsymbol{x}_i + \gamma_{3,1} h \boldsymbol{k}_1 + \gamma_{3,2} h \boldsymbol{k}_2 = \boldsymbol{x}_i + \begin{bmatrix} k_{1,1} & k_{1,2} \\ \vdots & \vdots \\ k_{n,1} & k_{n,2} \end{bmatrix} [h\gamma_{3,1} \quad h\gamma_{3,2}]^T$$

- Transposing the submatrix is necessary so that the dimensions fit. With $\mathbf{K} \in \mathbb{R}^{n \times 2}$ and $\boldsymbol{\Gamma}_{3,2} \in \mathbb{R}^{1 \times 2}$ the matrix multiplication yields a vector of dimension $\mathbb{R}^{n \times 1}$, i.e., the dimension of the state vector $\boldsymbol{x}_i$

Exercise 1: Solution

- For implementing this calculation and applying it to all stages, notice the indices of the slices
  - $\mathbf{K} \in \mathbb{R}^{n \times 4}$ for RK4 as four slopes are computed
  - For the third stage only the first two columns are needed (one less than the stage number)
  - $\boldsymbol{\Gamma}_{3,2} \in \mathbb{R}^{1 \times 2}$ in the third stage, so the submatrix is located in the row of the stage and contains two columns (again, one less than the stage number)
- The first step of an explicit Runge-Kutta method will always be a forward Euler method, calculating that step before starting the for-loop is recommended

Object-Oriented Programming in MATLAB

- MATLAB is capable of Object-Oriented Programming (OOP)
- This means classes can be defined with which objects are created (similar to Python, Java, C++, etc.)
- A class can contain properties (variables) and methods (functions) that are defined beforehand
- With a defined class multiple objects of the same type can be created
- Using OOP in MATLAB is more overhead than just using functions, but at the same time it offers more flexibility
- Defining a general class for a Runge-Kutta stepper is a good way to start

RKF78

- Implementing higher-order methods directly leads to long code sections just to define all equations with coefficients
- A flexible implementation will have more overhead, but can offer additional benefits
- How many stages (function evaluations) for the 7th and 8th order method?

RK 7(8)

κ \ λ	α_κ	$\beta_{\kappa\lambda}$ 0	1	2	3	4	5	6	7	8	9	10	11	c_κ	$\hat{c}_\kappa$
0	0	0												$\frac{41}{840}$	0
1	$\frac{2}{27}$	$\frac{2}{27}$												0	
2	$\frac{1}{9}$	$\frac{1}{36}$	$\frac{1}{12}$											0	
3	$\frac{1}{6}$	$\frac{1}{24}$	0	$\frac{1}{8}$										0	
4	$\frac{5}{12}$	$\frac{5}{12}$	0	$-\frac{25}{16}$	$\frac{25}{16}$									0	
5	$\frac{1}{2}$	$\frac{1}{20}$	0	0	$\frac{1}{4}$	$\frac{1}{5}$								$\frac{34}{105}$	
6	$\frac{5}{6}$	$-\frac{25}{108}$	0	0	$\frac{125}{108}$	$-\frac{65}{27}$	$\frac{125}{54}$							$\frac{9}{35}$	
7	$\frac{1}{6}$	$\frac{31}{300}$	0	0	0	$\frac{61}{225}$	$-\frac{2}{9}$	$\frac{13}{900}$						$\frac{9}{35}$	
8	$\frac{2}{3}$	2	0	0	$-\frac{53}{6}$	$\frac{704}{45}$	$-\frac{107}{9}$	$\frac{67}{90}$	3					$\frac{9}{280}$	
9	$\frac{1}{3}$	$-\frac{91}{108}$	0	0	$\frac{23}{108}$	$-\frac{976}{135}$	$\frac{311}{54}$	$-\frac{19}{60}$	$\frac{17}{6}$	$-\frac{1}{12}$				$\frac{9}{280}$	
10	1	$\frac{2383}{4100}$	0	0	$-\frac{341}{164}$	$\frac{4496}{1025}$	$-\frac{301}{82}$	$\frac{2133}{4100}$	$\frac{45}{82}$	$\frac{45}{164}$	$\frac{18}{41}$			$\frac{41}{840}$	0
11	0	$\frac{3}{205}$	0	0	0	0	$-\frac{6}{41}$	$-\frac{3}{205}$	$-\frac{3}{41}$	$\frac{3}{41}$	$\frac{6}{41}$	0			$\frac{41}{840}$
12	1	$-\frac{1777}{4100}$	0	0	$-\frac{341}{164}$	$\frac{4496}{1025}$	$-\frac{289}{82}$	$\frac{2193}{4100}$	$\frac{51}{82}$	$\frac{33}{164}$	$\frac{12}{41}$	0	1		$\frac{41}{840}$

[2]

RKF78

- For both 7th and 8th order it is more stages than the theoretical minimum
- Again, the benefit here is that it is an embedded method
- Finding coefficients for a method is cumbersome and highly mathematical
- For application it is easier to use a tested method with known coefficients

RK 7(8)

κ \ λ	α_κ	$\beta_{\kappa\lambda}$ 0	1	2	3	4	5	6	7	8	9	10	11	c_κ	$\hat{c}_\kappa$
0	0	0												$\frac{41}{840}$	0
1	$\frac{2}{27}$	$\frac{2}{27}$												0	
2	$\frac{1}{9}$	$\frac{1}{36}$	$\frac{1}{12}$											0	
3	$\frac{1}{6}$	$\frac{1}{24}$	0	$\frac{1}{8}$										0	
4	$\frac{5}{12}$	$\frac{5}{12}$	0	$-\frac{25}{16}$	$\frac{25}{16}$									0	
5	$\frac{1}{2}$	$\frac{1}{20}$	0	0	$\frac{1}{4}$	$\frac{1}{5}$								$\frac{34}{105}$	
6	$\frac{5}{6}$	$-\frac{25}{108}$	0	0	$\frac{125}{108}$	$-\frac{65}{27}$	$\frac{125}{54}$							$\frac{9}{35}$	
7	$\frac{1}{6}$	$\frac{31}{300}$	0	0	0	$\frac{61}{225}$	$-\frac{2}{9}$	$\frac{13}{900}$						$\frac{9}{35}$	
8	$\frac{2}{3}$	2	0	0	$-\frac{53}{6}$	$\frac{704}{45}$	$-\frac{107}{9}$	$\frac{67}{90}$	3					$\frac{9}{280}$	
9	$\frac{1}{3}$	$-\frac{91}{108}$	0	0	$\frac{23}{108}$	$-\frac{976}{135}$	$\frac{311}{54}$	$-\frac{19}{60}$	$\frac{17}{6}$	$-\frac{1}{12}$				$\frac{9}{280}$	
10	1	$\frac{2383}{4100}$	0	0	$-\frac{341}{164}$	$\frac{4496}{1025}$	$-\frac{301}{82}$	$\frac{2133}{4100}$	$\frac{45}{82}$	$\frac{45}{164}$	$\frac{18}{41}$			$\frac{41}{840}$	0
11	0	$\frac{3}{205}$	0	0	0	0	$-\frac{6}{41}$	$-\frac{3}{205}$	$-\frac{3}{41}$	$\frac{3}{41}$	$\frac{6}{41}$	0			$\frac{41}{840}$
12	1	$-\frac{1777}{4100}$	0	0	$-\frac{341}{164}$	$\frac{4496}{1025}$	$-\frac{289}{82}$	$\frac{2193}{4100}$	$\frac{51}{82}$	$\frac{33}{164}$	$\frac{12}{41}$	0	1		$\frac{41}{840}$

[2]

Remarks (1)

- A flexible implementations allows for different RK methods to be used interchangeably
- Different RK methods have different strengths, including, but not limited to:
  - Stability of the solution (robustness) vs efficiency of the method
  - Embedded or not embedded, effectively meaning more function evaluations than the theoretical minimum for the given order, but easy step size control
  - Interpolation
- Offering flexibility with your implementation offers methods of different strengths for the use of the methods

Remarks (2)

- Implementing in a flexible way (as you will have to do in the assignment) requires a level of understanding beforehand (as you have at least partially gained during this lecture)
- A class-based approach for implementing Runge-Kutta steppers is for example used in ESA's flight dynamics library GODOT written in C++
  - Here, a handful of different RK methods of different orders is offered
- MATLAB itself is not a compiled language, therefore it is comparatively slow

Step Size Control

- To reduce the computational effort of solving an IVP, the step size of a method can be adapted during the computation of the solution
- To be able to do this efficiently, an estimation of the error is necessary from which the change of the step size is derived

Step Size Control with Combined Approach (1)

- Step size control is often implemented using a combined approach
- Both EPS (absolute error) and REPS (relative error) are used to set the step size in an effective way
- h_i is to be set so that the error $\boldsymbol{e}_{i+1}$ is approximately equal to the specified tolerances
  - The relative error tolerance is used to calculate the relative error using $\boldsymbol{x}_i$ and $\boldsymbol{x}_{i+1}$

$$e_{i+1,j} \approx \epsilon_{abs} + \epsilon_{rel} \max\left(\left|x_{i,j}\right|, \left|x_{i+1,j}\right|\right)$$

- Important: This operation is done **element-wise**! The index j is the row here with $j = 1, \ldots, n$ and n as the dimension of the state vector, while i indicates the step and NOT a dimension
- For the error tolerances, use at least $\epsilon_{abs} = \epsilon_{rel} = 10^{-7}$ (going lower is also fine)

Step Size Control with Combined Approach (2)

- The error $\boldsymbol{e}_{i+1}$ is estimated using a second, different RK method of different order
  - With embedded methods like RKF45, either the 5th or the 4th order can be used for error estimation. We will use the 5th order method so that the 4th method can be used for calculating the next state.
  - The error is estimated as follows with $\boldsymbol{x}_{i+1}$ as the 4th order solution and $\hat{\boldsymbol{x}}_{i+1}$ as the 5th order solution:

$$\boldsymbol{e}_{i+1} \approx \boldsymbol{x}_{i+1} - \hat{\boldsymbol{x}}_{i+1} = h\left(\boldsymbol{\Phi} - \hat{\boldsymbol{\Phi}}\right)$$

Step Size Control with Combined Approach (3)

- The tolerances are combined using the solution of the method used to propagate (4th order in this case) to get ϵ:

$$\epsilon_j = \epsilon_{abs} + \epsilon_{rel} \max\left(\left|\hat{x}_{i,j}\right|, \left|\hat{x}_{i+1,j}\right|\right)$$

- With ϵ, the estimated error e_{i+1}, j is normalized:

$$e_{norm,j} = \left|\frac{e_{i+1,j}}{\epsilon_j}\right|$$

- Now the largest component of the normalized error $\boldsymbol{e}_{norm}$ is selected for subsequent use

$$e_{norm,max} = \max\left(\left|e_{norm,1}\right|, \ldots, \left|e_{norm,j}\right|, \ldots, \left|e_{norm,n}\right|\right)$$

Step Size Control with Combined Approach (4)

- The factor α is computed using a safety factor of 0.9 with the order of the method p:

$$\alpha = 0.9 \left(\frac{1}{e_{norm,max}} \right)^{\frac{1}{p+1}} = 0.9 \; e_{norm,max}{}^{-\frac{1}{p+1}}$$

- Using $e_{norm,max}$, check if the current step size h_i is acceptable
- **Case 1**: if $e_{norm,max} \leq 1$, the step size h_i for the current step is accepted and the step size $h_{i+1}$of the subsequent step is increased with $h_{i+1} = \alpha \; h_i$
- **Case 2**: if $e_{norm,max} > 1$, the step size h_i for the current step is not accepted. A new step size $h_{i,k}$ is computed by decreasing h_i with $h_{i,k} = \alpha \; h_i$ and the step is repeated

Step Size Control with Combined Approach (5)

- When computing a new step size, α can be limited to a specified maximum and minimum value
  - In GODOT these are 1.5 and 0.1 respectively
- This limit ensures that the step size does not increase or decrease too rapidly
- If an approach like this is implemented, the newly calculated step size has to be checked again to ensure that the error is tolerable

Step Size Control with Combined Approach (6)

- To start the integration, an initial step size has to be provided
- The initial step size should fit for the application and for orbit propagation an initial step size of 1 is good
- During the error estimation the computation can be simplified even further. To recall:

$$\boldsymbol{e}_{i+1} \approx \boldsymbol{x}_{i+1} - \hat{\boldsymbol{x}}_{i+1} = h\left(\boldsymbol{\Phi} - \hat{\boldsymbol{\Phi}}\right)$$

- Looking at $\boldsymbol{\Phi}$ and $\hat{\boldsymbol{\Phi}}$ one can see that all slopes $\boldsymbol{k}_i$ of the 4th order solution are the same as all but one slopes of the 5th order solution
- This means that computing a solution using the weights $\beta_i - \hat{\beta}_i$ immediately yields $\boldsymbol{e}_{i+1}$ which can be done in parallel to computing the 4th order solution for propagation

Step Size Control with Combined Approach (7)

- Using RKF45 as an example
- The increment function for propagating is using the weights in the last row
- The increment function for error estimation uses the differences of the 4th and 5th order weights so that

0	0	0	0	0	0	0	
$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$	0	0	0	0	zusätzliche Spalte bei RKF5
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$	0	0	0	
1	$\frac{439}{216}$	-8	$\frac{3680}{513}$	$-\frac{845}{4104}$	0	0	
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	0	zusätzliche Zeile bei RKF5
	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$		β_i für RKF4
	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$	β_i für RKF5

$$h\left(\boldsymbol{\Phi}-\hat{\boldsymbol{\Phi}}\right)$$

[1]

$$= h\left(\left(\beta_1-\widehat{\beta_1}\right)\boldsymbol{k}_1+\left(\beta_2-\widehat{\beta_2}\right)\boldsymbol{k}_2+\left(\beta_3-\widehat{\beta_3}\right)\boldsymbol{k}_3+\left(\beta_4-\widehat{\beta_4}\right)\boldsymbol{k}_4+\left(\beta_5-\widehat{\beta_5}\right)\boldsymbol{k}_5+\left(0-\widehat{\beta_6}\right)\boldsymbol{k}_6\right)$$

Step Size Control with Combined Approach (8)

- During each step, two increment functions are computed:
  - The increment function $\mathbf{\Phi}$ for the 4th order
  - The increment function $\left(\mathbf{\Phi} - \widehat{\mathbf{\Phi}}\right)$ for the error estimation
- Storing the weight differences $\beta_i - \widehat{\beta}_i$ directly is the easiest, there is no need for storing the 5th order weights independently

0	0	0	0	0	0	0	
$\frac{1}{4}$	$\frac{1}{4}$	0	0	0	0	0	
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$	0	0	0	0	zusätzliche Spalte bei RKF5
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$	0	0	0	
1	$\frac{439}{216}$	−8	$\frac{3680}{513}$	$-\frac{845}{4104}$	0	0	
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	0	zusätzliche Zeile bei RKF5
	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$		β_i für RKF4
	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$	β_i für RKF5

[1]

Image Index

[1] Hermann, Martin. „Numerik gewöhnlicher Differentialgleichungen Band 1: Anfangswertprobleme und lineare Randwertprobleme“, 2. ed., De Gruyter, 2017

[2] https://www.mathworks.com/matlabcentral/fileexchange/61130-runge-kutta-fehlberg-rkf78